



CSE 3433- SOFTWARE ARCHITECTURE

Lecturer: Dr Mohamad Nor Bin Hasan

Topic: Event-Driven Job Recruitment & Application
Processing

System (Event-Driven Architecture)

Group: Group 12

No	Name	Matric Number
1	MUHAMAD NUR IMAN HAFIZI BIN MOHD ADIB	S75659
2	MUHAMMAD HARITH RIEZHAN BIN SYANAZ	S75804
3	MUHAMMAD SYUKRAN BIN MOHD ZAKRI	S75572

Table of Contents

1. PROJECT PROPOSAL.....	2
1.1 SELECTED ARCHITECTURE STYLE	3
1.2 PROBLEM DESCRIPTION	3
1.3 SYSTEM OBJECTIVES.....	3
1.4 INITIAL FUNCTIONAL REQUIREMENTS	5
1.5 TEAM MEMBER ROLES	5
2. PROJECT PROPOSAL.....	5
2.1 Functional Requirement.....	6
2.2 Non-functional Requirement	6
2.3 Architecture Style Explanation	7
2.4 Service or Component Identification	8
2.5 Architecture Diagram	10
2.5.1 System Context Diagram.....	11
2.5.2 Component/ Service Architecture Diagram	11
2.5.3 Interaction Diagram.....	12
2.5.4 Deployment Diagram.....	13
3. Technology Stack.....	13
4. Prototype Implementation.....	15
4.1 Tool Used	16
4.2 User Interface.....	16
5. Architecture Evaluation	16
5.1 Evaluation Method	17
5.2 Evaluation Results	17
5.3 Strengths and Limitations	18
6. Team Contributions	18
7. Conclusion and Reflection	19
7.1 Conclusion	20
7.2 Reflection	20
References	20

1. PROJECT PROPOSAL

1.1 SELECTED ARCHITECTURE STYLE

To efficiently manage communication and processing between different components of the system, the Event-Driven Job Recruitment & Application Processing System adopts an Event-Driven Architecture (EDA) style. This architecture allows system components to communicate through events that are generated whenever an important action occurs within the system. In this approach, different services operate independently but respond to specific events triggered by user actions.

In this system, user such as employers and job applicants interacts with the platform through the user interface. When employer posts a job vacancy or when an applicant submits a job application, an event is triggered and sent to the event processing mechanism.

Using Event-Driven Architecture allows the system to achieve loose coupling between components, improving scalability, flexibility, and maintainability when handling multiple recruitment activities. Therefore, the proposed system applies Event-Driven Architecture to support efficient recruitment processing, automated workflows, and scalable system performance. or instance, once an application is submitted, a screening service may automatically evaluate the resume, while a notification service sends confirmation messages to the applicant. This parallel processing significantly reduces delays and improves overall system efficiency.

1.2 PROBLEM DESCRIPTION

Many organizations receive many job applications when the advertise job vacancies. However, the recruitment process is often handled using traditional or centralized systems that process tasks sequentially. This approach can cause delays in processing applications, difficulties in tracking applicant status, and inefficiencies when managing multiple recruitment activities at the same time. However, in many cases, the recruitment process is still handled using traditional or centralized systems that process tasks sequentially.

Besides, applicants may not receive timely updates regarding their application status, while recruiters may struggle to manage tasks such as screening resumes, shortlisting candidates and sending notifications efficiently. Without an automated and responsive system, the recruitment process can become slow and difficult to manage. Therefore, an event-driven recruitment system is needed to automate workflows and improve the efficiency of handling job applications. f one part of the system becomes slow or fails it can disrupt the entire recruitment workflow. This lack of responsiveness makes it difficult for organizations to adapt quickly to changing hiring needs or unexpected increases in application volume.

1.3 SYSTEM OBJECTIVES

1. To develop a centralized system that allows employers to post job vacancies and applicants to view available job opportunities efficiently.
2. To implement an application management mechanism that supports job application submission, tracking, and status updates.
3. To design the system using an event-driven approach, ensuring loose coupling and efficient communication between system components.
4. To provide a notification feature that keeps applicants informed about their application status and recruitment progress.
5. To improve recruitment efficiency by automating application processing and managing recruitment activities in a more organized and systematic way.

1.4 INITIAL FUNCTIONAL REQUIREMENTS

For the initial functional requirement of the event-driven Job Requirement and Application processing system that we will focus on the main activities of both candidates and recruiters with the interaction between services. First, a system needs to support the efficient job and candidate management which is the recruiters need a reliable way to post and provide all the detail about the job vacancies. Candidates play a role as a simple job to create profile, upload resume and apply the jobs of their capabilities so the system exist because of both parties which employers and job seekers get their require to centralized and structured environment of their jobs to interact with.

Besides, the system needs to handle entire recruitment workflow smoothly for real time which including submission application, shortlisting, interview scheduling and decision making for recruiter to hire the candidates which is appropriate with the jobs. In an event-driven, especially important because each step depend on events being processed quickly and accurately to reduce delays and ensure every action trigger immediately as soon as the system receive the notification.

Furthermore, crucial part of the system needs to provide communication and decision support to the candidates because they expect time updates about application status need to insight into hiring process and performance. This needs a notification system and basic analytics or reporting features to ease the candidate for manually checked on their application submission on their jobs. These ensure that users stay informed and organizations can improve their hiring strategies based on user's data to make sure the system is not just functional, but also responsive, scalable and user-focused to what they need.

1.5 TEAM MEMBER ROLES

Name	Roles
MUHAMAD NUR IMAN HAFIZI BIN MOHD ADIB	Database Lead
MUHAMMAD HARITH RIEZHAN BIN SYANAZ	Backend Lead
MUHAMMAD SYUKRAN BIN MOHD ZAKRI	Frontend Lead

2. PROJECT PROPOSAL

2.1 Functional Requirement

1. Job Management

- i. The system shall allow recruiters to create, update and delete job postings.
- ii. The system shall allow recruiters to define job details such as title, description, required skills, salary and location.
- iii. The system shall allow recruiters to publish and close job vacancies.

2. Candidate Management

- i. The system shall allow candidates to register and create profiles.
- ii. The system shall allow candidates to update personal information, skills, experience and manage their resumes.

3. Job Search and Discovery

- i. The system shall allow candidates to search for jobs using filters such as skills, location, and job type.
- ii. The system shall provide relevant job listings based on search queries.

4. Job Application Submission

- i. The system shall allow candidates to apply for job postings, attach resumes and document's support.
- ii. The system shall prevent duplicate job applications.

5. Recruitment Workflow Processing

- i. The system shall allow recruiters to review submitted applications.
- ii. The system shall allow recruiters to shortlist or reject candidates.
- iii. The system shall allow recruiters to schedule interviews.

6. Event-Driven Processing

- i. The system shall generate events for key actions such as job posting, application submission, and status updates.
- ii. The system shall trigger appropriate actions automatically upon receiving events.

7. Notification System

- i. The system shall notify recruiters when a new application is submitted.
- ii. The system shall notify candidates of any changes in application status.

2.2 Non-functional Requirement

1. Performance

- i. The system shall process events within 1–5 seconds.
- ii. The system shall return job search results within 2 seconds under normal conditions.

2. Scalability

- i. The system shall support horizontal scaling to handle users and workloads.
- ii. The system shall handle a high volume of concurrent job applications and events.

3. Availability

- i. The system shall maintain at least 99.9% uptime.
- ii. The system shall remain operational during partial service failures.

4. Reliability

- i. The system shall ensure that no critical events are lost.
- ii. The system shall implement retry mechanisms for failed event processing.

5. Consistency

- i. The system shall maintain eventual consistency across distributed services.
- ii. The system shall ensure data accuracy after event processing is completed.

6. Security

- i. The system shall enforce authentication and authorization mechanisms.
- ii. The system shall implement role-based access control for users.
- iii. The system shall protect sensitive data using encryption and secure communication.

7. Fault Tolerance

- i. The system shall continue operating despite failures in individual services.
- ii. The system shall implement error handling and recovery mechanisms.

8. Maintainability

- i. The system shall be modular and based on microservices architecture.
- ii. The system shall allow independent updates and deployment of services.

9. Usability

- i. The system shall provide an intuitive and user-friendly interface.
- ii. The system shall ensure ease of use for both candidates and recruiters.

2.3 Architecture Style Explanation

The Event-Driven Job Recruitment & Application Processing System is designed using an Event-Driven Architecture (EDA). This architectural style enables different system components to communicate through events that are triggered when specific actions occur within the system. Instead of tightly coupling system components together, EDA allows services to operate independently while responding to events asynchronously. This approach is suitable for systems that require real-time processing and flexible workflows such as job recruitment platforms.

In this system, employers and job applicants interact with the platform through the user interface. When a user performs an action, such as posting a job vacancy or submitting a job application, the system generates an event. The event is then sent to an event processing mechanism that distributes it to the relevant services. For example, when a job application is submitted, an `ApplicationSubmittedEvent` is triggered. This event can be handled by multiple services such as the Application Management Service, Resume Screening Service, and Notification Service, which process the application and update the system accordingly.

Using Event-Driven Architecture provides several advantages such as loose coupling, improved scalability, and asynchronous processing. These characteristics allow the system to handle multiple recruitment activities efficiently without affecting other system operations. The system is also divided into independent services such as Job Posting Service, Application Management Service, Resume Screening Service, and Notification Service. Each service listens to specific events and performs its designated task.

Therefore, the use of Event-Driven Architecture allows the recruitment system to process events efficiently, support automated workflows, and handle many job applications in a scalable and flexible manner.

2.4 Service or Component Identification

The Event-Driven Job Recruitment & Application Processing System consists of several independent services and components that work together to support the recruitment process. Each component has a specific responsibility and communicates through events using the event bus. The following components are identified in the system architecture.

1. Job Portal / Mobile Application

This component represents the user interface where candidates and recruiters interact with the system. Recruiters can post job vacancies, while candidates can search for jobs and submit applications through the portal.

2. API Gateway

The API Gateway acts as the entry point for all client requests. It receives requests from the job portal or mobile application and routes them to the appropriate backend services. It also helps manage request handling and communication between the client and services.

3. Job Management Service

This service is responsible for managing job vacancy information. It allows recruiters to create, update, and manage job postings. When a new job is created, the service generates a `JobCreatedEvent` that is sent to the event bus.

4. Application Service

The Application Service handles job application submission from candidates. It records application details and stores them in the application database. When an application is submitted, an `ApplicationSubmittedEvent` is triggered and published to the event bus.

5. Event Bus

The Event Bus acts as the central communication mechanism for the event-driven system. It receives events from services and distributes them to other services that subscribe to those events. Technologies such as Kafka or RabbitMQ can be used to implement this component.

6. Resume Parsing Service

This service processes applicant resumes automatically by extracting important information such as skills, education, and work experience. It is triggered when an application submission event occurs.

7. Notification Service

The Notification Service sends notifications to candidates and recruiters through email or SMS. It is triggered by events such as application submission, status updates, or interview scheduling.

8. Search and Indexing Service

This component manages job search functionality by indexing job postings and allowing candidates to quickly search for relevant job opportunities.

9. Analytics and Reporting Service

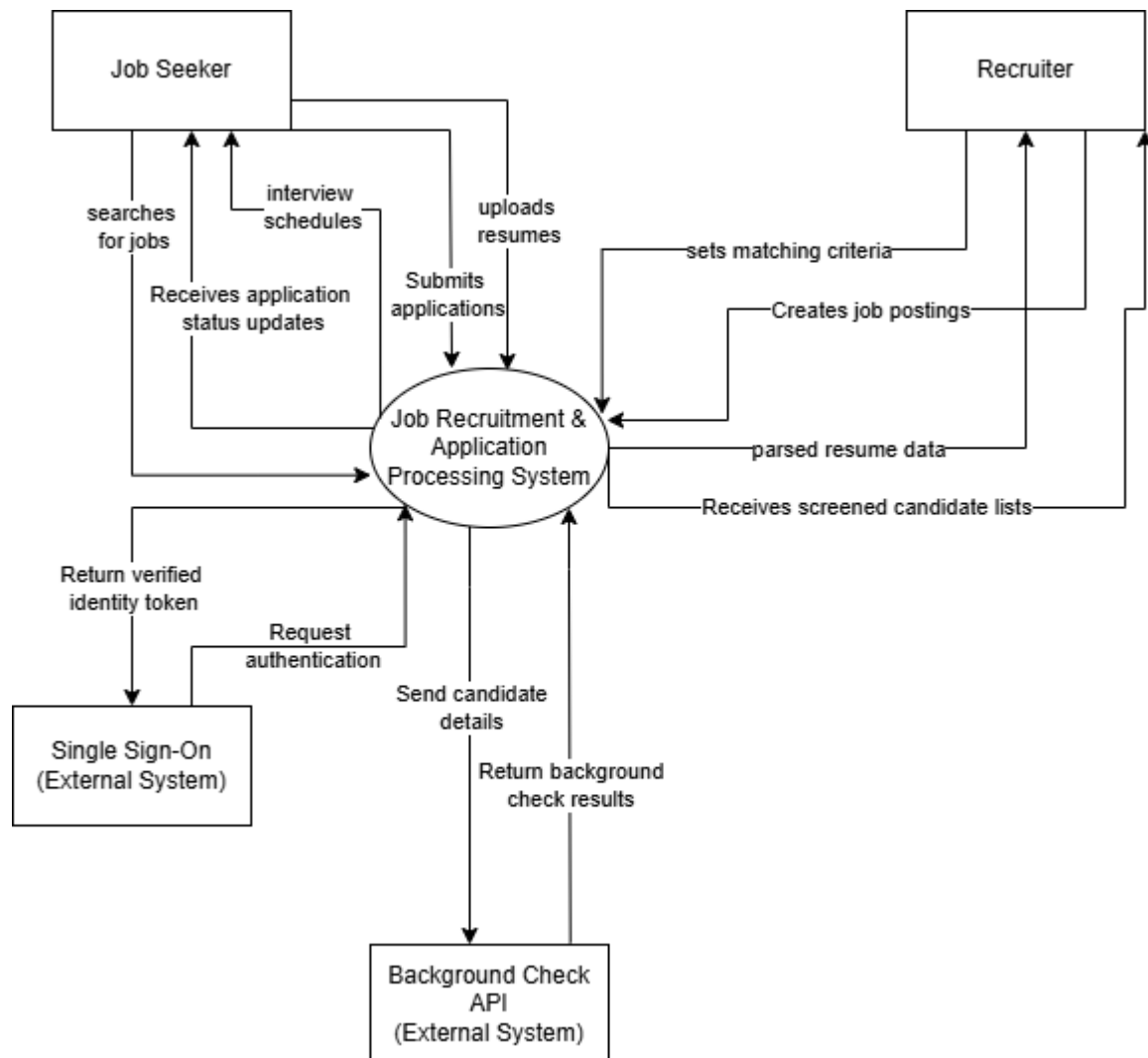
This service analysed recruitment data and generated reports such as the number of applications received, recruitment progress, and hiring statistics.

10. Background Check Service

The Background Check Service performs verification of applicant information if required by the recruiter, such as employment history or credential validation.

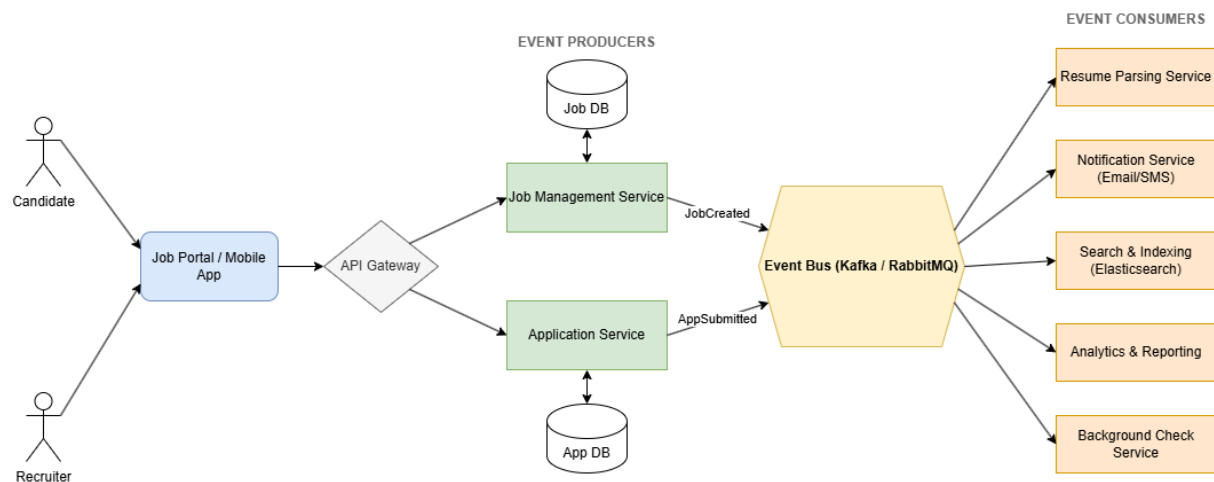
2.5 Architecture Diagram

2.5.1 System Context Diagram



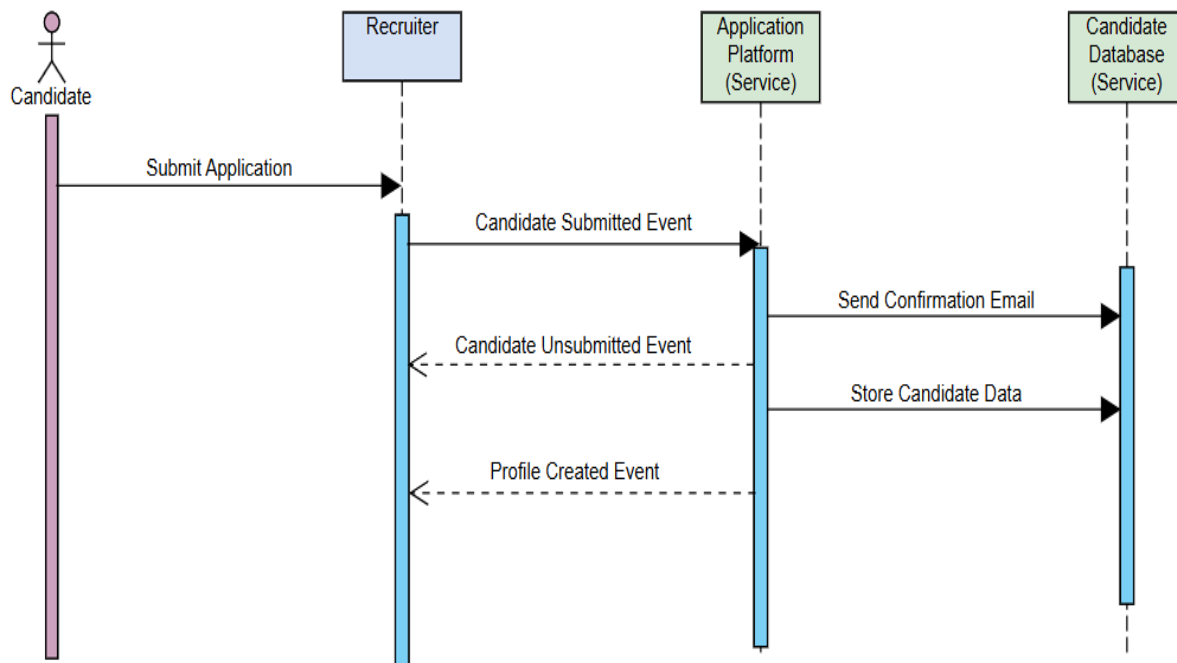
The system context diagram illustrates the Job Recruitment & Application Processing System as the central platform that connects job seekers, recruiters, and external services. Job seekers interact with the system by searching for jobs, submitting applications, uploading resumes, and receiving application status updates or interview schedules. Recruiters use the system to create job postings, define candidate matching criteria, and receive screened candidate lists based on processed resume data. The system also integrates with external services, including a Single Sign-On (SSO) system for user authentication and a Background Check API to verify candidate information. Overall, the system acts as a hub that manages the flow of information between users and external systems to streamline the recruitment process.

2.5.2 Component/ Service Architecture Diagram



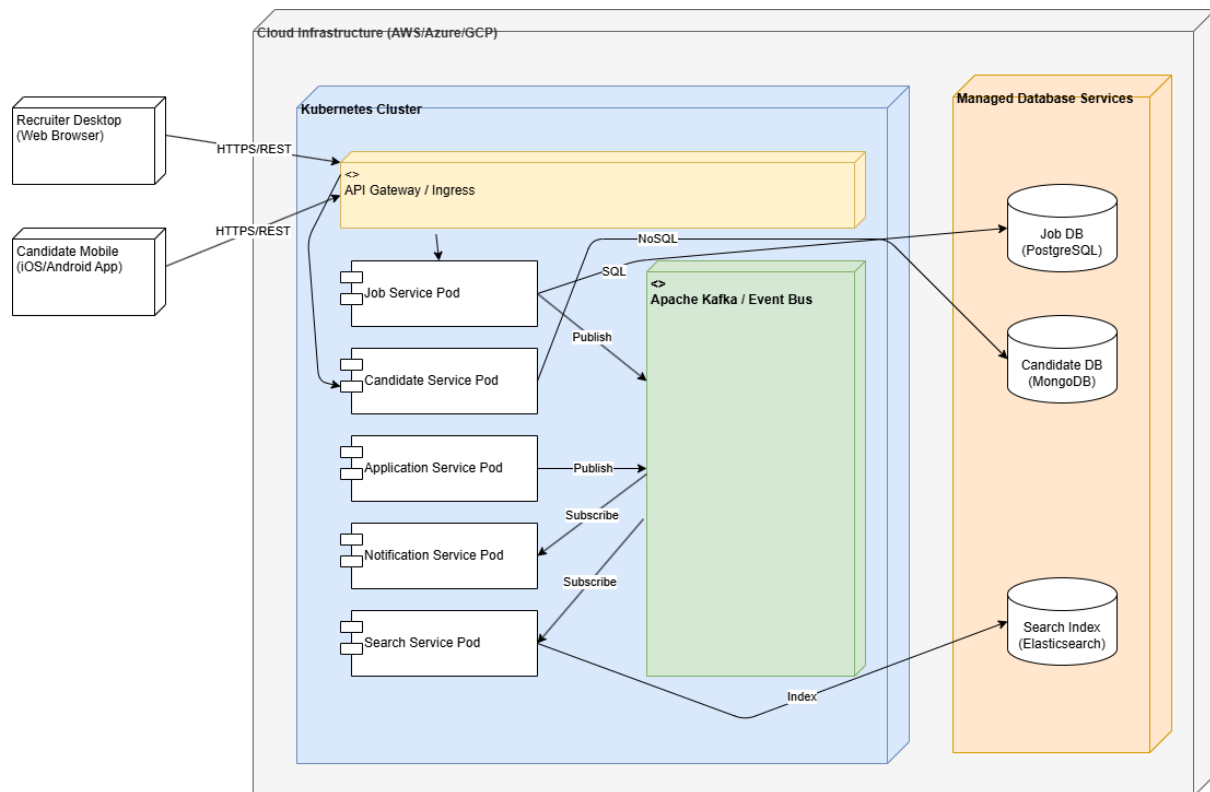
This Component / Service Architecture diagram illustrates an event-driven architecture for the Job Recruitment System, where candidates and recruiters interact through a Job Portal or Mobile App that communicates via an API Gateway. The system is divided into core services, including the Job Management Service and Application Service, each connected to their respective databases. When key events occur, such as job creation or application submission, they are published to an Event Bus such as Kafka or RabbitMQ. These events are then consumed by various independent services, including resume parsing, notification (email/SMS), search and indexing, analytics, and background check services. This architecture enables loose coupling, scalability, and real-time processing by allowing multiple services to react to events asynchronously without directly depending on each other.

2.5.3 Interaction Diagram



This Interaction Diagram outlines the process for submitting a job application. It begins with the Candidate submitting an application through the Application Platform, which then generates both a Candidate Submitted Event and a Profile Created Event. In parallel, the platform triggers a Send Confirmation Email action to the candidate and stores the candidate's data in the Candidate Database. If the candidate does not complete the submission, a Candidate Unsubmitted Event is recorded. Overall, the diagram shows how the system handles successful submissions, data storage, and email notifications, while also tracking incomplete applications.

2.5.4 Deployment Diagram



This deployment diagram illustrates a cloud-based job application platform running on infrastructure such as AWS, Azure, or GCP. At the front end, a Candidate Mobile app communicates via HTTPS/REST through an API Gateway or Ingress into a Kubernetes cluster. Inside the cluster, separate service pods handle job, application, notification, and search functions. These services communicate asynchronously through an Apache Kafka event bus and interact with managed database services, including a PostgreSQL job database, a MongoDB candidate database, and an Elasticsearch index for search. The architecture ensures scalability, real-time event-driven processing, and separation of data storage types based on use case.

3. Technology Stack

Architecture Layer	Technology Used	Technical Purpose & Choice Justification
Presentation Layer	HTML5 / CSS3 / JavaScript (with JSP)	Handles the user system interface for both candidate and recruiter portals. It generates dynamic, data-driven web layouts directly from the backend server to keep dashboard views synchronized with state changes.
Web Controller / Event Producer	Java Servlets (Java EE)	Acts as the front door for incoming HTTP client requests. It intercepts user submissions (like posting a job or applying), immediately validates form data payloads, and generates the corresponding system event without blocking the client.
Message Broker / Event Engine	Custom Singleton EventDispatcher	Manages the internal asynchronous message queues within a single runtime instance. Utilizing thread pools and background executors, it immediately buffers and distributes system events to independent consumer threads.
Data Access Object (DAO)	Java DAO Pattern (with JDBC)	Powers database interactions across the modular backend. It completely decouples business logic workflows from physical relational data storage mechanisms, ensuring clean data access layers.

Database Layer	MySQL Server	Serves as the central relational storage system. It maintains individual data tables for jobs, candidate profiles, and application logs, enforcing eventual consistency across the different decoupled modules.
Media Storage	Cloudinary API	Handles heavy user-uploaded file artifacts, specifically candidate resume documents. Offloading static document assets to a managed storage provider keeps the primary MySQL database light and highly responsive.
Local Web Container	Apache Tomcat	Provides the localized web server environment to execute our compiled servlets and host application contexts, maintaining stable routing configurations.

4. Prototype Implementation

This section details the system implementation, core event engine loops, and client-facing interfaces of the Event-Driven Job Recruitment & Application Processing System prototype. The architecture highlights a decoupled setup written in Java EE (Servlets, JSP) and managed through a relational MySQL database server. Under this Event-Driven Architecture (EDA) paradigm, the system leverages front-facing web controllers as event producers. These controllers capture incoming client actions and publish request payloads asynchronously to a central, thread-pooled EventDispatcher.

The developed prototype establishes two primary user roles: Candidate and Recruiter, each configured with specific workflow paths and data permissions. Candidates can create personal accounts, update their profiles, browse through active job advertisements, upload resume files, and submit applications. Recruiters, on the other hand, are equipped to create and publish new job vacancies, review incoming applicant submissions, track candidates across the recruitment pipeline, and modify active application states throughout the asynchronous system cycle.

4.1 Tool Used

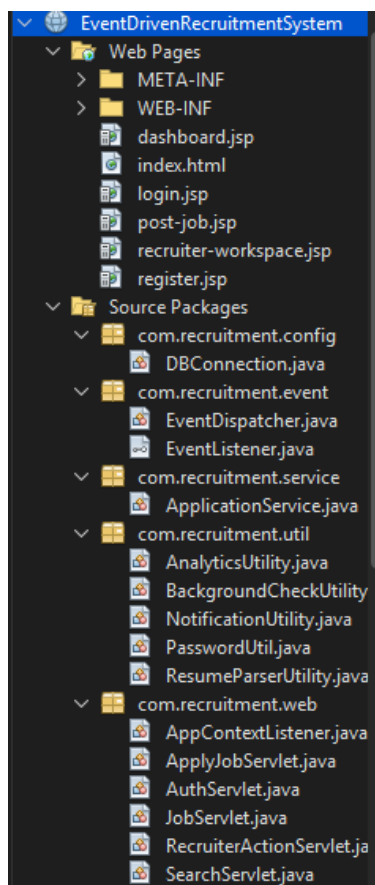


Diagram 4.1.1: Structure of file used in Apache NetBeans to implement frontend and backend

The project follows a layered Java web application architecture organized under the **com.recruitment** package hierarchy. The backend is structured into four main packages: config, which handles database connectivity through DBConnection.java event, which provides the core Event-Driven Architecture (EDA) infrastructure with EventDispatcher.java serving as the central asynchronous event router and EventListener.java as the observer interface service which encapsulates business logic in ApplicationService.java and implements the transactional outbox pattern and util which contains asynchronous event consumers such as **NotificationUtility**, **ResumeParserUtility**, **BackgroundCheckUtility**, **AnalyticsUtility**, and **PasswordUtil**. The web package contains servlets including **AuthServlet**, **ApplyJobServlet**, **JobServlet**, and **SearchActionServlet**, which act as HTTP entry points, as well as **AppContextListener**, which registers event listeners with the dispatcher during application startup. The frontend is implemented using JSP views such as **dashboard.jsp**, **recruiter-workspace.jsp**, **post-job.jsp**, **login.jsp**, and **register.jsp**. Providing role-based interfaces for candidates and recruiters. Data persistence is managed through a MySQL database (recruitment.db) that stores both domain data and event logs. This modular structure demonstrates a clean event-driven architecture where HTTP requests trigger domain operations, generate events, and dispatch them asynchronously to loosely coupled consumer utilities, improving scalability, maintainability, and system flexibility.

Table	Action	Rows	Type	Collation	Size	Overhead
applications	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	48.0 KiB	-
event_log	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	16.0 KiB	-
jobs	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	64.0 KiB	-
users	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_general_ci	32.0 KiB	-
4 tables	Sum	1	InnoDB	utf8mb4_general_ci	160.0 KiB	0 B

Diagram 4.1.2: Database that implemented in phpmyadmin MySQL]

The system uses a MySQL relational database consisting of four main tables: users, jobs, applications, and event log. The users table stores recruiter and candidate information, jobs manage job vacancies, applications records job submissions, and event log tracks system events generated during recruitment activities. The database supports the Event-Driven Architecture by maintaining structured recruitment data, ensuring data integrity through relational constraints, and providing reliable storage for system operations.

4.2 User Interface

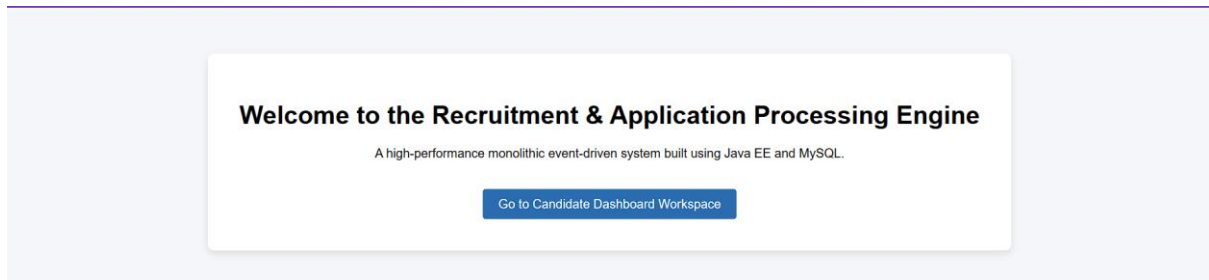
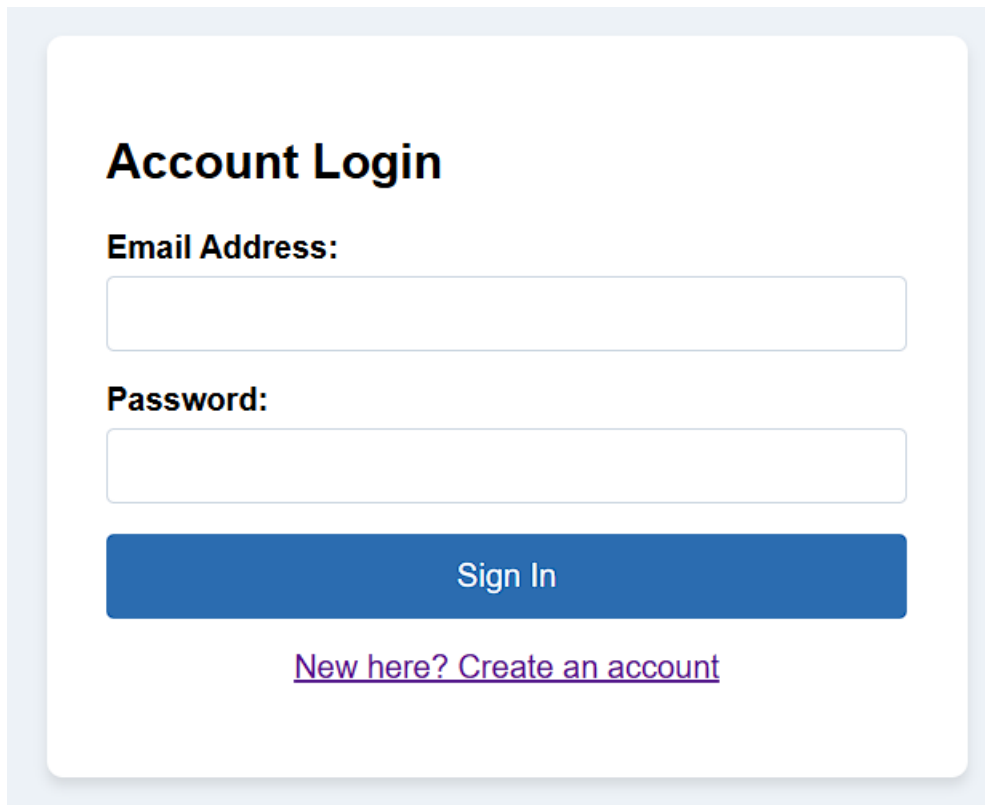


Diagram 4.2.1: Homepage of recruitment & Application Processing System

The registration form is titled "Create an Account" in bold black font. It contains four input fields: "Full Name:", "Email Address:", "Password:", and "Account Role:". The "Account Role:" field is a dropdown menu with "Candidate (Job Seeker)" selected and a downward arrow. Below the fields is a large blue "Sign Up" button. At the bottom, there is a link that says "Already have an account? Login" in purple text.

Diagram 4.2.2: Registration Form



The diagram shows a login form with a title, two input fields, a button, and a link.

Account Login

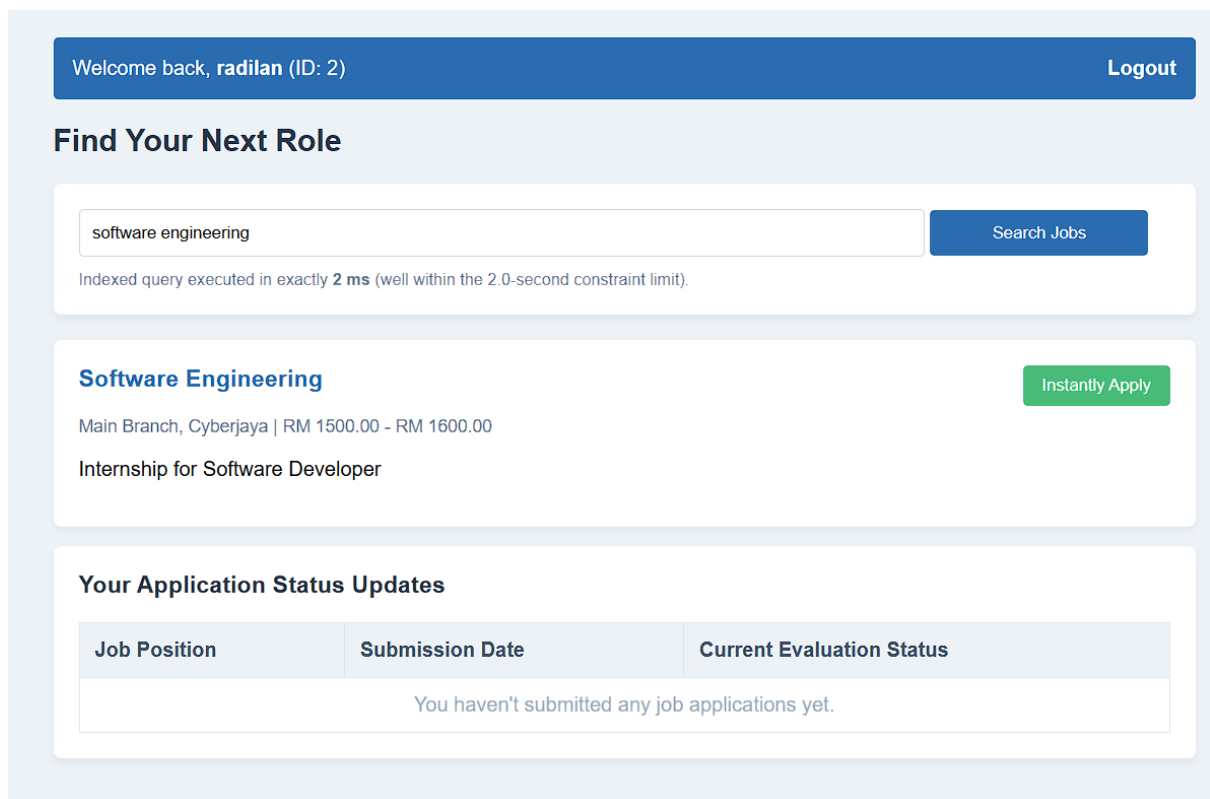
Email Address:

Password:

Sign In

[New here? Create an account](#)

Diagram 4.2.3: Login Form



The diagram shows a candidate page with a header, a search section, a job listing, and an application status section.

Welcome back, **radilan** (ID: 2) Logout

Find Your Next Role

software engineering Search Jobs

Indexed query executed in exactly **2 ms** (well within the 2.0-second constraint limit).

Software Engineering

Main Branch, Cyberjaya | RM 1500.00 - RM 1600.00

Internship for Software Developer Instantly Apply

Your Application Status Updates

Job Position	Submission Date	Current Evaluation Status
You haven't submitted any job applications yet.		

Diagram 4.2.4: Candidate Page

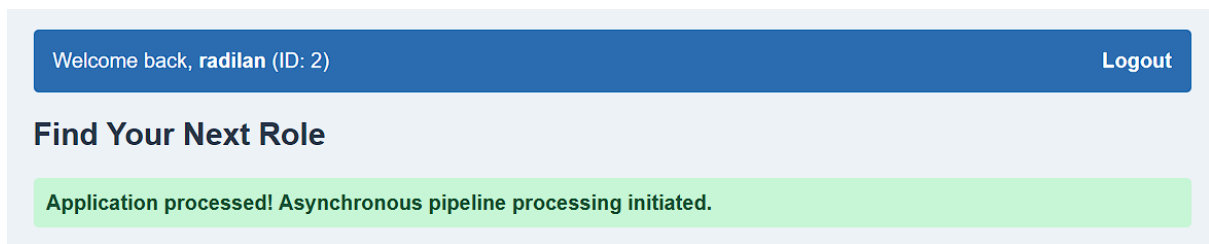


Diagram 4.2.5: Notification Successful applied

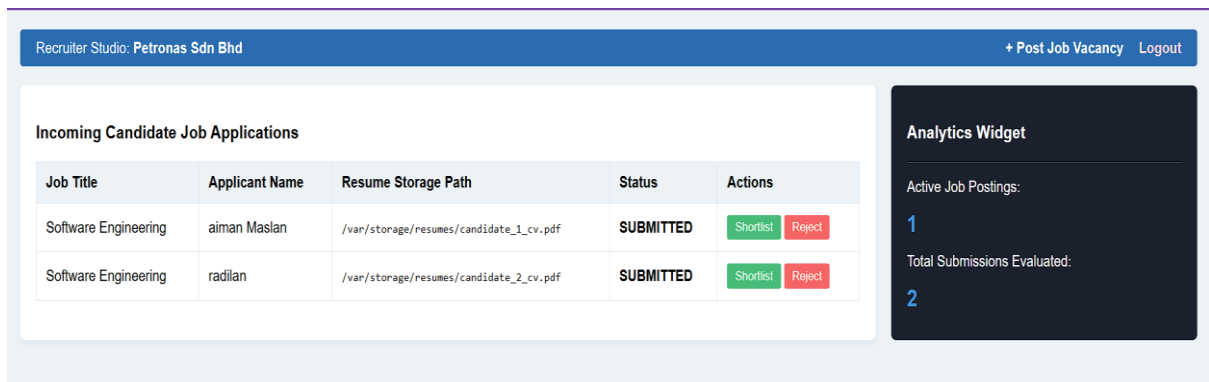


Diagram 4.2.6: Recruiter Page

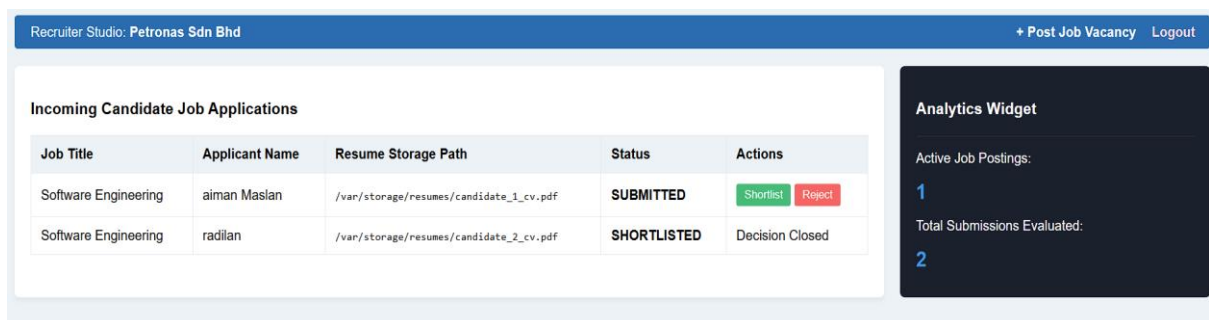


Diagram 4.2.7: Respond when recruiter accept the application

5. Architecture Evaluation

5.1 Evaluation Method

The architecture of the Event-Driven Job Recruitment & Application Processing System was evaluated based on key quality attributes including scalability, performance, availability, maintainability, security, and reliability. The evaluation focuses on how well the selected Event-Driven Architecture (EDA) supports the system requirements and business objectives. The assessment was conducted by analysing the architecture design, service interactions, and event-processing workflow.

5.2 Evaluation Results

1. Scalability

The Event-Driven Architecture supports horizontal scalability by allowing services to operate independently. Components such as the Job Management Service, Application Service, Notification Service, and Resume Parsing Service can be scaled separately according to workload demands. During periods of high application volume, additional service instances can be deployed without affecting other components. This makes the system suitable for organizations experiencing large numbers of job applications.

2. Performance

The architecture improves system performance through asynchronous event processing. When a candidate submits a job application, the event is published to the Event Bus and processed by multiple services simultaneously. For example, resume parsing, notifications, and analytics updates can occur in parallel, reducing response time and improving user experience. This design supports the non-functional requirement of processing events within a few seconds.

3. Reliability and Fault Tolerant

The Event Bus acts as a central communication mechanism that ensures events are delivered to subscribed services. If one service temporarily fails, other services can continue operating without disruption. Retry mechanisms and message persistence can be implemented within technologies such as Kafka or RabbitMQ to prevent event loss and improve reliability. This increases the overall fault tolerance of the recruitment platform.

4. Maintainability

The system follows a modular service-based structure where each service has a specific responsibility. Developers can modify, update, or deploy individual services without affecting the entire system. This reduces maintenance complexity and supports future enhancements such as adding AI-based candidate screening or advanced analytics services.

5. Security

Security is supported through the API Gateway, authentication mechanisms, and role-based access control. Sensitive user information such as candidate profiles and resumes can be protected through encrypted communication and secure storage. The separation of services also reduces the impact of potential security breaches by limiting access to specific system components.

6. Flexibility

One of the mayor strengths of Event-Driven Architecture is flexibility. New services can subscribe to existing events without requiring modifications to existing components. For example, a future interview Recommendation Service or AI Candidate Matching Service can be added by subscribing to application related events. This enables continuous system evolution with minimal disruption.

5.3 Strengths and Limitations

Strengths

- Supports real-time and asynchronous processing.
- Provides loose coupling between services.
- Improves scalability and system flexibility.
- Enables independent deployment and maintenance of services.
- Handles high volumes of concurrent job applications efficiently.

Limitations

- Increased architectural complexity compared to traditional monolithic systems.
- Event tracking and debugging can be more difficult due to distributed processing.
- Requires additional infrastructure such as Kafka or RabbitMQ.
- Eventual consistency may cause temporary data synchronization delays between services.

Overall, the Event-Driven Architecture is highly suitable for the Job Recruitment & Application Processing System. The evaluation shows that the architecture effectively supports scalability, performance, reliability, and maintainability requirements while providing flexibility for future expansion. Although the architecture introduces additional complexity, the benefits significantly outweigh the challenges, making it an appropriate architectural choice for a modern recruitment platform.

6. Team Contributions

Name	Matrics Number	Contribution and Responsibility
MUHAMAD NUR IMAN HAFIZI BIN MOHD ADIB	S75659	i) Managed the integrated ElasticSearch of search engine to making easier for search queries of the job. ii) Managed MySQL Database structure and user data storage. iii) Database handled by validate and localized client-side form structure to prevent double data inserted. iv) Manage and handle all the project report are written in the right format and divide the module for each team members.
MUHAMMAD HARITH RIEZHAN BIN SYANAZ	S75804	i) Analysed and built core microservices of high-level system architecture using Node.js ii) Defined and format all the logic handler for each module in the system. iii) Handle and model the system behaviour by defining all diagram constructed for the system process.
MUHAMMAD SYUKRAN BIN MOHD ZAKRI	S75572	i) Analysed and implemented all the Functional Requirement and Non-Functional Requirement that related to the system. ii) Designed user system-interface by using React Native for candidate job discovery. iii) Managed frontend development and integrate communication due to the protocol to pull the data from API Gateway.

7. Conclusion and Reflection

7.1 Conclusion

The Event-Driven Job Recruitment & Application Processing System was successfully designed to address common challenges in traditional recruitment processes, including delays in application handling, inefficient communication, and difficulties in managing large volumes of job applications. By adopting Event-Driven Architecture (EDA), the proposed system enables different services to communicate asynchronously through events, allowing recruitment activities to be processed efficiently and independently.

The system architecture consists of several interconnected services, including Job Management Service, Application Service, Resume Parsing Service, Notification Service, Search and Indexing Service, Analytics and Reporting Service, and Background Check Service. These services collaborate through an Event Bus, enabling automated workflows and reducing dependencies between system components. The architecture promotes scalability, flexibility, fault tolerance, and maintainability, making it suitable for modern recruitment environments.

Throughout the project, functional and non-functional requirements were identified, architecture diagrams were developed, suitable technologies were selected, and a prototype implementation was proposed. The architecture evaluation further demonstrated that Event-Driven Architecture effectively supports the system's requirements by improving responsiveness, supporting high volumes of concurrent activities, and enabling future system expansion. Therefore, the proposed architecture provides a strong foundation for developing a modern, efficient, and scalable recruitment platform.

7.2 Reflection

This project provided valuable exposure to the software architecture design process and the importance of selecting an appropriate architectural style based on system requirements and business objectives. Through the development of the Event-Driven Job Recruitment & Application Processing System, the team gained practical experience in analysing problems, gathering requirements, designing system architectures, and evaluating architectural decisions.

One of the key lessons learned was the significance of loose coupling and asynchronous communication in modern distributed systems. Studying and applying Event-Driven Architecture allowed the team to better understand how independent services can collaborate efficiently through events while maintaining scalability and flexibility. The project also enhanced the team's understanding of architectural modelling through the creation of system context diagrams, component diagrams, interaction diagrams, and deployment diagrams.

In addition to technical knowledge, the project strengthened teamwork, communication, and project management skills. Each team member contributed according to their assigned responsibilities while working collaboratively to ensure consistency between requirements, architecture design, technology selection, and documentation. Challenges encountered throughout the project, such as defining service boundaries, designing event flows, and evaluating architecture quality attributes, provided valuable learning opportunities and encouraged critical thinking.

Overall, this project successfully achieved its intended learning outcomes and provided a deeper understanding of software architecture principles and practices. The knowledge and experience gained throughout this project will serve as a strong foundation for future software engineering projects and real-world system development.

References

1. Richardson, C. (2017). *Microservices.io*. Microservices.io; Chris Richardson. <https://microservices.io/patterns/data/event-driven-architecture.html>
2. GeeksforGeeks. (2024, January 31). *EventDriven Architecture System Design*. GeeksforGeeks. <https://www.geeksforgeeks.org/system-design/event-driven-architecture-system-design/>
3. *What is event-driven architecture? A complete guide*. (2025, September 11). Technology & Software Development Blog | Future Processing. <https://www.future-processing.com/blog/event-driven-architecture-guide/>
4. sallam, mohammed. (2025, August 26). *Fundamentals of software architecture Chapter 14: Event-Driven Architecture Style*. Medium. <https://medium.com/@mohamedsallam953/fundamentals-of-software-architecture-chapter-14-event-driven-architecture-style-ffea4ecbbce1>
5. Navarro, J., & Parrend, P. (2018). *Clustering analysis classes to identify software component*, 76. <https://www.sciencedirect.com/science/article/abs/pii/S095058491400130X>
6. *What is Deployment Diagram?* (n.d.). <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-deployment-diagram/>